

Exception Handling in Spring Boot — Quick Revision Notes

Overview

A professional API shouldn't just return data — it should clearly tell the client what happened (success, client error, missing resource, server error). This is done using HTTP status codes, `ResponseEntity`, custom exceptions, and global exception handling.

1. API Response: Communication Between Two Systems

Every API response has 3 parts: **Status Code** (meaning at HTTP level), **Headers** (metadata like `Location`, `Content-Type`), and **Body** (actual data/error message).

2. Beginner API vs Professional API

A beginner API only asks "did I get data or not?" A professional API asks "did the server clearly tell the client what happened?" — covering success, not-found, invalid input, and server errors separately.

3. Example: Professional API Responses

Same endpoint can have different outcomes: student exists (200 OK), doesn't exist (404), invalid request (400), or server error (500) — each needs its own proper response.

4. Common Problems in Beginner CRUD APIs

Beginners often return `200 OK` for everything (even failures), which isn't expressive. Better mapping: create → 201, not found → 404, validation fail → 400, duplicate → 409, backend issue → 500.

5. What Happens When an Exception Occurs

Normal flow: Client → Tomcat → DispatcherServlet → Controller → Service → Repository → Database. If an exception is thrown in the service (e.g., calling `.get()` on an empty `Optional`), it bubbles back up until something handles it.

6. Spring Boot Default Error Handling

Spring Boot has a built-in `/error` mapping — browsers see a Whitelabel Error Page, Postman gets a JSON response. By default it hides exception details for safety (can be enabled via `spring.web.error.include-message=always`, but not recommended in production).

7. Problem with Default Error Response

The default handler often returns a generic `500 Internal Server Error` even for normal cases like "student not found" — which is misleading. That case should actually be a `404`

`Not Found`). This is why custom exception handling is needed.

8. HTTP Status Codes for REST APIs

Status codes are grouped: 1xx = Informational, 2xx = Success, 3xx = Redirection, 4xx = Client error, 5xx = Server error. For most CRUD APIs: 200, 201, 204, 400, 404, 409, 500 are enough.

9. Common Status Codes in CRUD APIs

Fetch/update success → 200, resource created → 201, delete success (no body) → 204, invalid input → 400, resource not found → 404, duplicate data → 409, unexpected failure → 500.

10. 201 Created

Used when a new resource is successfully created (e.g., `POST /api/students`). Tells the client the resource now exists.

11. 204 No Content

Used when a request succeeds but there's nothing to return in the body — most common for DELETE APIs.

12. 409 Conflict

Used when the request is valid, but it conflicts with existing data — e.g., trying to create a student with an email that already exists.

13. 500 Internal Server Error

Used for unexpected backend failures (DB connection issue, `NullPointerException`, bugs) — NOT for normal business cases like "not found."

14. Why Use `ResponseEntity`?

Without `ResponseEntity`, Spring auto-decides the status code (usually 200). With `ResponseEntity`, you explicitly control the status code, headers, and body.

15. Mental Model of `ResponseEntity`

`ResponseEntity` can represent Body+Status, Body+Headers+Status, or Status only — e.g., `ResponseEntity.ok(student)` (200+body), `ResponseEntity.status(CREATED).body(x)` (201+body), `ResponseEntity.noContent().build()` (204, no body).

16. Returning Headers with `ResponseEntity`

For create APIs, it's useful to return a `Location` header pointing to the newly created resource, using `ResponseEntity.created(location).body(savedStudent)`.

17. Common ResponseEntity Methods

`ResponseEntity.ok(x)` = 200 + body. `ResponseEntity.noContent().build()` = 204 + no body (typical for DELETE APIs).

18. Problem with Handling Errors in Controller

Once beginners learn `ResponseEntity`, they start cramming all error-checking logic (null checks, status decisions) into the controller — making it messy. A controller should only: receive request → call service → return response.

19. Example: Duplicate Email Check in Controller

Putting business checks like "does this email already exist" directly in the controller leads to repeated if-else blocks across many APIs — not clean design.

20. Problems Caused by Repeated Error Logic

Leads to: repetition, unreadable controllers full of if-else, weak signals from the service layer (returning `null`), inconsistent error responses across APIs, and poor separation of concerns.

21. Better Separation of Responsibility

Clean design: **Controller** receives request & returns success response, **Service** runs business logic and throws meaningful exceptions, **Global Exception Handler** converts exceptions into proper HTTP responses.

22. What Is Global Exception Handling?

Instead of handling errors separately in every controller, create one central place that catches exceptions and converts them into consistent API responses.

23. What Is @ControllerAdvice?

An annotation that lets one class provide common behavior (like exception handling) across multiple controllers. By itself it does nothing — needs `@ExceptionHandler` methods inside it to actually catch exceptions.

24. What Is @RestControllerAdvice?

A shortcut combining `@ControllerAdvice` + `@ResponseBody` — so exception handler methods return JSON directly. Preferred for REST APIs.

25. What Is @ExceptionHandler?

Placed on a method to handle a specific exception type — e.g.,

`@ExceptionHandler(RuntimeException.class)`. Spring picks the most specific matching

handler when multiple exist.

26. Creating a Global Exception Handler Class

A class annotated with `@RestControllerAdvice` becomes the central place for handling all API errors.

27. Handling Generic Exceptions

A handler for `Exception.class` acts as a safety net for any unexpected/unhandled error — should NOT be used for normal business cases like "not found" (those need specific handlers).

28. Why Create Custom Exceptions?

Generic exceptions like `RuntimeException` don't clearly describe the business meaning. A custom exception like `ResourceNotFoundException` clearly communicates intent.

29. Creating ResourceNotFoundException

A custom exception class extending `RuntimeException` (unchecked, so no need to declare `throws` everywhere) — simple and practical for Spring Boot service-layer errors.

30. Throwing Custom Exception from Service Layer

Instead of returning `null` when data isn't found, the service should throw a `ResourceNotFoundException` using `.orElseThrow(...)` — much clearer than a silent `null`.

31. Clean Controller After Using Custom Exception

Once the service throws proper exceptions, the controller becomes clean — no `if`, no `null` check, no manual 404 logic. It just calls the service and returns the response.

32-33. Update & Delete API Examples

Both follow the same pattern: service fetches the record via `.orElseThrow(...)`, updates/deletes it, and the controller stays clean with just service call + response.

34. Handling Custom Exception Globally

`GlobalExceptionHandler` catches `ResourceNotFoundException` and returns a proper `404 Not Found` with the exception's message — much better than returning raw strings.

35. Why Use an Error Response DTO?

The client doesn't understand Java exceptions — it understands HTTP/JSON. So exceptions should be converted into a structured, standard error response instead of a plain string.

36. ErrorResponseDto Structure

A DTO containing `timestamp`, `statusCode`, `error`, `message`, and `path` — gives a consistent, professional error format for every failure.

37. Meaning of Each Error Field

`timestamp` helps match errors with logs. `statusCode` lets the frontend programmatically react (e.g., `if error.statusCode === 404`). `path` tells which endpoint failed — useful when a page calls multiple APIs.

38. Getting Request Path Using `HttpServletRequest`

`request.getRequestURI()` gives the current API path, which is included in the error response's `path` field.

39–40. Handling Validation Errors Globally

Request DTOs use annotations like `@NotBlank`, `@Size`, `@Email`, `@Min`. Validation only runs when the controller uses `@Valid @RequestBody`. If validation fails, Spring throws `MethodArgumentNotValidException`.

41. What Is `MethodArgumentNotValidException`?

Thrown when validation fails on a `@Valid` request body — the exception that triggers field-level error handling.

42–43. Field-Level Validation Error Response & DTO

A good validation error response includes a `fieldErrors` map showing exactly which field failed and why (e.g., `"name": "Name cannot be blank"`). Modeled using a `ValidationErrorResponseDto`.

44. Extracting Field Errors

Use `ex.getBindingResult().getFieldErrors()` to loop through failed fields and build a `Map<String, String>` of field name → error message.

45. Handler for Validation Errors

An `@ExceptionHandler(MethodArgumentNotValidException.class)` method builds the field-error map and returns a `400 Bad Request` with a `ValidationErrorResponseDto`.

46. What Is 400 Bad Request?

Means the client sent something wrong — includes validation failures, invalid JSON, wrong data types, malformed body, invalid path variables, or missing query params.

47-48. Invalid JSON / Wrong Data Type

If the client sends a wrong data type (e.g., `"age": "twenty"` instead of a number), Spring throws `HttpMessageNotReadableException` — handled to return a clean `400 Bad Request`.

49. Invalid Path Variable Type

If a path variable can't be converted (e.g., `/students/abc` where an ID is expected), Spring throws `MethodArgumentTypeMismatchException` — also handled as `400 Bad Request`.

50. Difference Between 400 and 409

400 = the request itself is invalid (bad format/type). **409** = the request is valid, but conflicts with existing data (like a duplicate email).

51-53. DuplicateResourceException

A custom exception (extends `RuntimeException`) thrown from the service when duplicate data is detected (e.g., existing email) — handled globally to return `409 Conflict`.

54. Complete GlobalExceptionHandler Class

Combines all handlers in one class: `ResourceNotFoundException` → 404, `DuplicateResourceException` → 409, `MethodArgumentNotValidException` → 400 (with field errors), `HttpMessageNotReadableException` → 400, `MethodArgumentTypeMismatchException` → 400, and generic `Exception` → 500 (fallback).

55. Categories of Exceptions

Three types: **Business exceptions** (`ResourceNotFound` → 404, `DuplicateResource` → 409), **Request/Input exceptions** (validation, bad JSON, type mismatch → 400), and **Unexpected exceptions** (generic fallback → 500).

56. Clean API Flow After Global Exception Handling

Request → Controller → Service → (success: return response) OR (failure: throw exception) → `GlobalExceptionHandler` catches it → proper error DTO returned to client.

57. Why This Makes APIs Professional

Controllers stay clean, no repeated error logic, service layer communicates failures clearly, frontend gets consistent responses, status codes match real meaning, and debugging is easier (timestamp + path in every error).

58. Best Practices

- Don't return `null` for missing resources — throw a custom exception instead.
- Keep success logic in the controller.

- Keep business checks (duplicate, not found, etc.) in the service.
- Keep error-response-building logic in the `GlobalExceptionHandler`.
- Use specific exception handlers first; generic handler is only a last-resort safety net.
- Never expose raw internal exception messages in production — use a safe generic message instead.

Final Summary

A professional Spring Boot API should clearly communicate both success and failure: use `ResponseEntity` for success responses (status + headers + body), and for failures — service throws meaningful custom exceptions, `GlobalExceptionHandler` catches them, and the API returns consistent HTTP error responses. This keeps the backend clean and easy for frontend developers to work with.